Townlight Notice Manual Beta 1

Windows x64 Beta 1 · MSI 1.8.1.1

Windows 1.8.1.1 adds shared meeting imports for 31 source families, with 34 supported source/format pairs. Installed Windows Sandbox checks exercised preview, save and durable readback using public captures and fixtures, plus the documented Townlight handoffs. These checks do not establish automatic discovery on every live portal or a complete retest of every module. Acceptance and limits.

- Townlight Notice User Manual
  - Windows staff workflow — 0.3.0
  - Standalone package boundary
  - For Non-Technical Users
  - For IT and Technical Staff
  - Start and Smoke-Check Townlight Notice
  - Architecture

Townlight Notice User Manual

Windows staff workflow — 0.3.0

Use a fresh Windows profile. The approved profile path is `%LOCALAPPDATA%\CivicSuite`. Keep existing profiles and backups.

1. Sign in with an account allowed to use Notice and open Public Notices. Create a draft with title, actual notice text, type, event date, staff-selected calendar lead days and staff-verified statutory basis. Save it. Its deadline workpaper uses the shared Core reminder helper; it does not establish legal sufficiency.
2. Select the saved notice, read the deadline workpaper and source, enter review notes, then record staff review. Approve that saved revision only after the responsible staff member has checked the law, timing, content and channels. Mechanical rule success never removes the staff-review requirement.
3. Perform official publication outside Townlight. Retain the actual proof file (up to 5 MB each and 10 MB total for the current approved content), posting location and actual posting date/time including time zone. A filename, target system name or checklist alone is not posting proof.
4. Create the archive after approval and proof. The archive contains the actual notice, retained proof bytes, review/approval history and a per-file SHA-256/size manifest. Write and verify the ZIP export to preserve a usable file. ZIP entry timestamps are fixed for deterministic packaging; the record contains the recorded posting and audit times.
5. For a Records handoff, enter the intended Records request UUID and select Attach archive to Records. This requires Records authority as well as Notice authority.

> Townlight retains an exact managed copy, verifies its hash, attaches it through the canonical Records writer and reports the request/archive identity and hash. Repeating the same handoff does not create a duplicate attachment. Open the request in Records and read back the attached archive. Disclosure still requires Records review/release authority.

Saved notices and history can be reopened from Public Notices. A content edit clears the prior review, approval and current proof selection; historical events/proof/archives remain retained. A stale open revision is rejected so staff must refresh and review the saved content. Missing review, approval or actual proof blocks the corresponding step. Back up/restore the matching complete new-format profile through System Health; the Windows acceptance summary records the installed backup/restore and archive readback checks.

This replaces the disconnected standalone Public Notices controls. The separate Meetings notice/template/proof workflow remains available for meeting-specific records; it does not silently approve a different Notice draft. Staff remain responsible for legal sufficiency, official publication and the official record. No subscriber message or official notice is sent automatically.

Standalone package boundary

The standalone FastAPI routes below remain registry/deadline/proof workpapers and deterministic planning helpers. The Windows authenticated lifecycle above is provided by the suite runtime using Notice's durable workflow store; it is not a claim that every standalone helper performs an archive or Records transfer. Fixed rule packs are review aids and are not administered jurisdiction/effective-date legal rules. Subscriber/channel/accessibility/export helpers retain their documented planning boundaries.

For Non-Technical Users

Townlight Notice helps city staff organize public hearing notices, legal notices, bid notices, vacancy notices, publication deadlines, channel planning notes, accessibility and language-readiness checks, proof requirements, subscriber delivery plans, archive/handoff packets, and export manifests. It can create a sample notice registry stub, build publication deadline reminders, check a draft packet against deterministic staff-review rules, create a staff-editable notice template, retrieve saved registry/deadline/publication-proof workpapers when IT enables persistence, assemble publication-readiness checklists, summarize channel planning, build accessibility/plain-language/translation readiness packets, build subscriber delivery plans, assemble archive/handoff packets, and assemble a notice/records export checklist.

Notice 0.3.0 ships in the Windows x64 beta 1.8.1.1 with Townlight Core 1.3.0.dev2. The Windows acceptance summary covers the installed staff workflow. Standalone deployment is documented separately below.

For IT and Technical Staff

Townlight Notice is a FastAPI Python package pinned to the `townlight-core 1.3.0.dev2` release wheel. The current runtime exposes:

Set `TOWNLIGHT_NOTICE_WORKPAPER_DB_URL` to enable SQLAlchemy-backed notice registry, deadline-plan, and publication-proof records. Leave it unset for deterministic stateless operation:

registry and deadline POST requests return transient payloads, persisted GET routes return actionable 503 responses, and publication-proof storage is unavailable. When persistence is enabled, set `TOWNLIGHT_NOTICE_TRUSTED_WRITE_TOKEN` and send the matching `X-Townlight-Notice-Write-Token` header for durable write routes.

- `GET /`
- `GET /health`
- `GET /townlight_notice`
- `GET /docs`
- `GET /openapi.json`
- `POST /api/v1/townlight_notice/registry`
- `GET /api/v1/townlight_notice/registry/{record_id}`
- `POST /api/v1/townlight_notice/deadlines`
- `GET /api/v1/townlight_notice/deadlines/{plan_id}`
- `POST /api/v1/townlight_notice/rule-check`
- `POST /api/v1/townlight_notice/templates`
- `POST /api/v1/townlight_notice/publication-proof`
- `GET /api/v1/townlight_notice/publication-proof/{proof_id}`
- `POST /api/v1/townlight_notice/publication-check`
- `POST /api/v1/townlight_notice/accessibility-review`
- `POST /api/v1/townlight_notice/channels`
- `POST /api/v1/townlight_notice/subscribers/plan`
- `POST /api/v1/townlight_notice/archive-packet`
- `POST /api/v1/townlight_notice/export`

Start and Smoke-Check Townlight Notice

A fresh user can reach the core stateless API and public sample UI without a database, model server, account, or API key. Start the ASGI app target `townlight_notice.main:app`, then confirm `/health` reports Townlight Notice `0.3.0` and Townlight Core `1.3.0.dev2`.

Minimal smoke-check workflow:

1. Create a transient registry stub with `/api/v1/townlight_notice/registry`.
2. Open `/docs` or `/openapi.json` to inspect accepted fields, then create a deterministic rule check with `/api/v1/townlight_notice/rule-check`. Unsupported notice types return supported choices instead of silently falling back to a generic rule.
3. Open `/townlight_notice` and confirm it is a static public sample with boundary copy.
4. If durable workpapers are enabled, confirm the trusted write token is configured before testing persistence-backed writes.

Run:

```
python -m pip install -e ".[dev]"
python -m pytest -q
bash scripts/verify-release.sh
```

Architecture

```
flowchart LR
  Staff["Clerk / communications / purchasing staff"] --> Townlight
Notice["Townlight Notice"]
  Townlight Notice --> Townlight Core["Townlight Core 1.3.0.dev2"]
  Townlight Notice -. future handoff .-> Townlight Meetings["Townlight
Meetings v0.1.0"]
  Townlight Notice -. future handoff .-> CivicProcure["CivicProcure v0.1.1"]
  Townlight Notice --> Export["Notice and records export checklist"]
```

Townlight Notice depends on Townlight Core. Townlight Core does not depend on Townlight Notice. Townlight Notice v0.3.0 uses the shared Townlight Core notice deadline helper plus deterministic sample notice data, staff-review rule packs, notice templates, accessibility and language-readiness packets, subscriber delivery plans, archive/handoff packets, and optional publication-proof workpapers only; live agenda/procurement handoffs, legal sufficiency decisions, legal advice, official notice publication, publication-system write-back, and production notice-system integrations are future work.

Shared meeting imports · Windows 1.8.1.1

Windows 1.8.1.1 adds shared meeting imports for 31 source families, with 34 supported source/format pairs. Installed Windows Sandbox checks exercised preview, save and durable readback using public captures and fixtures, plus the documented Townlight handoffs. These checks do not establish automatic discovery on every live portal or a complete retest of every module.

All 31 sources, formats, operator steps and integration contracts · Installed acceptance and limits.

Windows 1.8.1.1 has fresh-install acceptance. An in-place upgrade to 1.8.1.1 was not exercised. The previous Windows 1.8.0.1 release tested upgrade from 1.7.2.1; that result applies to 1.8.0.1. Before an existing-city evaluation, back up city data and settings, close Townlight, keep the existing %LOCALAPPDATA%\CivicSuite profile, and use the same Windows account. Keep the prior installer and a restore path. Station continues to use its separately configured service.

The accepted MSI contains Townlight Runtime 0.3.1 and Meetings 1.0.6 distributions. Its health version strings still report Runtime 0.3.0 and Meetings 1.0.5. Default-branch source corrects those display strings; the signed MSI and its exact build source remain unchanged.

Shared meeting imports

Use **Meetings → Import meeting records** in Windows 1.8.1.1. Select a source and format, enter the original public URL, then choose **Preview import**. For a saved response, select the file before previewing. Choose a meeting and use **Save selected meeting**. Townlight records the source URL, format, stable source ID and capture hash with the canonical meeting. Reopening

the same capture returns the existing meeting. A changed capture for that identity is reported for staff review.

The source and format selector lists the supported reader pairs below. A public URL must serve the chosen format directly. Use a saved public response for POST-only listings, portals that require a browser session and sources whose page does not return the required response. The importer reads what a municipality publishes in that response. Fields and attachments vary by municipality and by meeting.

Source	Input to preview
Legistar	Core-format JSON export
PrimeGov	Core-format JSON export or public portal meeting list JSON
Granicus	Core-format JSON export or public ViewPublisher HTML
NovusAGENDA	Core-format JSON export
CivicPlus Agenda and Meeting Management Select	Public Events JSON
CivicPlus Agenda Center	Public listing HTML
Municode Meetings	Public meetings HTML
BoardBook	Public organization HTML
eSCRIBE	Public calendar JSON
Simbli	Public meeting listing HTML
iCompass CivicWeb	Public meeting-type listing HTML
iQM2	Public meeting list HTML
Granicus Swagit	Public video archive HTML
SuiteOne Media	Public meeting archive HTML
Granicus Peak	Public agenda center ViewPublisher HTML
Utah Public Notice Website	Public notice detail HTML
Destiny AgendaQuick	Public meeting list HTML
Town Hall Streams	Public archive HTML
TelVue	Public playlist HTML
CHAMP	Public event JSON
Hyland OnBase	Public Agenda Online HTML
Laserfiche WebLink	Saved public dated document folder HTML
TownCloud	Saved public agenda table JSON or public agenda detail HTML
Digital Towpath	Public meeting detail HTML

Source	Input to preview
ClerkBase	Public meeting document HTML
Municipality-published calendar	Public iCalendar feed
Wistia	Public media JSON
Boardable	Public Site meeting detail HTML
gWorks FrontDesk	Public city meetings JSON
Diligent BoardDocs	Unencrypted printed agenda PDF with numbered items
Tyler Meeting Manager	Saved public calendar JSON. Meeting information and item titles when present.

Staff handoffs

- **Core and Meetings.** Core's existing vendor formats share the same preview and save path as the public readers. The saved meeting and any structured agenda items open in the regular Meetings workflow.
- **Records.** In Meetings, load imported documents by meeting ID, choose a PDF and enter the destination Records request ID. The attachment retains the citation and file hash. An HTML viewer or a dead document link may need staff to obtain the PDF separately.
- **Notice and Access.** Use **Review imported text** to start a Notice draft or accessibility review with the source citation. Use each workspace's normal review and save actions.
- **Station.** In the configured Station operator page, select the saved meeting and stage a direct MP4. Use the private capture workflow to queue processing. Player-only references are visible without a staging action.
- **Code.** Use the existing ordinance proposal and adoption workflow with the canonical imported meeting and agenda IDs.
- **Boards, Contracts, Procurement, Zoning, Planning, Permitting and Inspections.** In the destination workspace use **Imported meeting evidence**. Select an existing staff record and the saved meeting, then save the citation. The destination's existing role and workflow apply.

Shared interface

The installed Townlight Python runtime exposes a read-only CLI. CivicCast and other local clients can consume its normalized JSON and apply their own access and persistence rules.

```
python -m townlight_runtime.meeting_imports champ PUBLIC_EVENT_URL saved-event.json --format champ-event-json
```

Use `--fetch` in place of the saved file to read a supported public response.

Interfaces and other products

The version-1 normalized JSON contract is shared by the Python function `townlight_runtime.meeting_imports.preview_export(connector, source_url,`

`content, input_format=...)` and the CLI above. A saved response can also be read from stdin using `-`. Inputs are limited to 16 MiB. Preview is read-only; the consuming application owns persistence, staff authorization and its own data.

The installed native HTTP bridge exposes `GET /desktop/v1/meetings/imports/sources`, `POST /desktop/v1/meetings/imports/preview`, `POST /desktop/v1/meetings/imports/fetch`, `POST /desktop/v1/meetings/imports/commit`, and meeting/link/document handoff operations in the native API schema. Use the existing `X-Townlight-Session` authentication, mapped staff roles and configured local PostgreSQL service. This is a native application interface; there is no independent anonymous importer HTTP service or importer MCP server.

CivicCast: opt in through its Townlight shared importer source setting and `TOWNLIGHT_IMPORT_PYTHON` (the absolute Python executable), `TOWNLIGHT_IMPORT_CONNECTOR`, `TOWNLIGHT_IMPORT_FORMAT`, `TOWNLIGHT_IMPORT_URL` and optional `TOWNLIGHT_IMPORT_CAPTURE`. The adapter saves normalized items through the existing AgendaStore. Structured agenda availability depends on the source response; a video listing can contain meeting metadata without agenda items. Existing sources remain the default.

CivicCast Brief: its existing PrimeGov and Granicus discovery routes can opt into the shared reader using `TOWNLIGHT_IMPORT_PYTHON`. Brief does not expose all 31 sources as a new selection menu. Its existing caption/media workflow remains the consuming path. See each project's shared-import client documentation.

TownReporter, Describer Workbench and Textflowkit: no shared-import adapter was added to these products. TownReporter retains its own PrimeGov path. Describer Workbench and Textflowkit expose their existing transcription APIs/MCP interfaces, which do not constitute meeting-import integration. The shared engine ships inside Townlight Runtime; a standalone library is outside this release.

Windows acceptance and limits

Windows 1.8.1.1 adds shared meeting imports for 31 source families, with 34 supported source/format pairs. Installed Windows Sandbox checks exercised preview, save and durable readback using public captures and fixtures, plus the documented Townlight handoffs. These checks do not establish automatic discovery on every live portal or a complete retest of every module.

Windows 1.8.1.1 has fresh-install acceptance. An in-place upgrade to 1.8.1.1 was not exercised. The previous Windows 1.8.0.1 release tested upgrade from 1.7.2.1; that result applies to 1.8.0.1. Before an existing-city evaluation, back up city data and settings, close Townlight, keep the existing %LOCALAPPDATA%\CivicSuite profile, and use the same Windows account. Keep the prior installer and a restore path. Station continues to use its separately configured service.

The accepted MSI contains Townlight Runtime 0.3.1 and Meetings 1.0.6 distributions. Its health version strings still report Runtime 0.3.0 and Meetings 1.0.5. Default-branch source corrects those display strings; the signed MSI and its exact build source remain unchanged.

Exact Windows 1.8.1.1 acceptance and limits.